



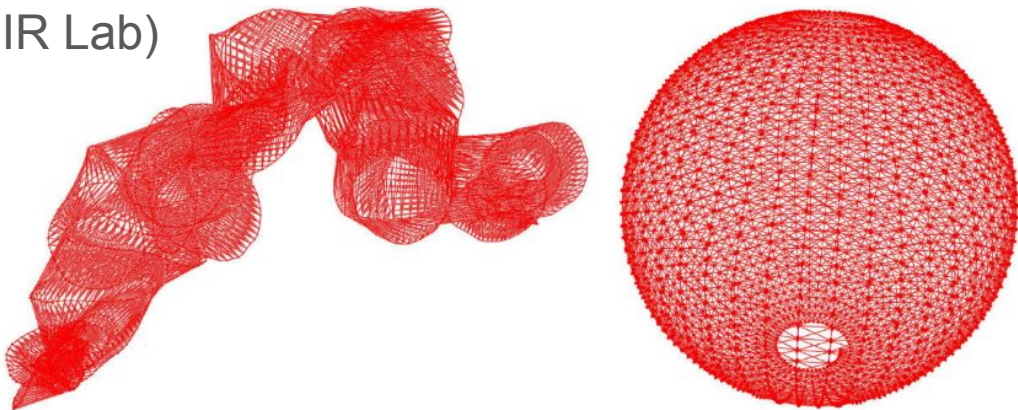
# Mobile Robotics

## 3D Graph SLAM

---

Socially Intelligent Robot (SIR Lab)  
University of Stuttgart  
Prof. Dr. Kai Arras

Annie Bhalla  
Likhith Jain





# 3D Graph SLAM

### State Spaces

*Robot:* Translation -  
x,y,z & Rotation -  
roll, pitch, yaw  
*Landmarks:* x,y,z

Graph Based  
Representation  
Pose Graphs -  
Nodes, Edges

Simultaneous  
Localization  
And  
Mapping



## Contents

- Introduction to SLAM
- Key Concepts of 3D Graph SLAM
- Graph SLAM Optimization
- SLAM Optimization in 2D
- SLAM Optimization in 3D



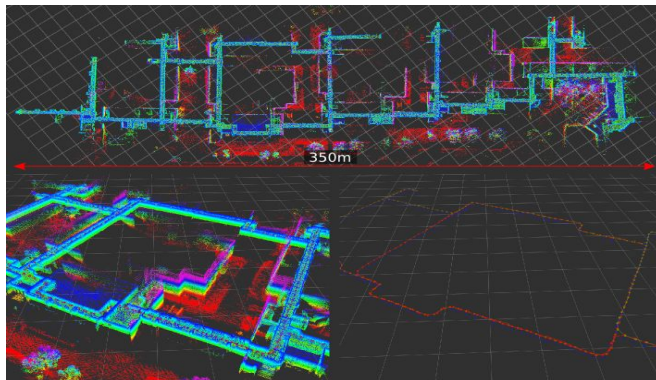
## Contents

- **Introduction to SLAM**
- Key Concepts of 3D Graph SLAM
- Graph SLAM Optimization
- SLAM Optimization in 2D
- SLAM Optimization in 3D



## What is SLAM ?

- Computing the robot's poses and the map of the environment at the same time
- **Localization:** Estimating robots location
- **Mapping:** Building a map
- **SLAM:** Simultaneous Localization and Mapping





## Probabilistic Formulation of SLAM

- Solving the SLAM problem consists of estimating the posterior probability of the robots trajectory  $\mathbf{x}_{0:T}$ , and the map  $\mathbf{m}$ , of the environment given all the measurements.

$$p\left(\mathbf{x}_{0:T}, \mathbf{m} \mid \mathbf{z}_{1:T}, \mathbf{u}_{1:T}\right)$$

robot trajectory

map

observations

controls



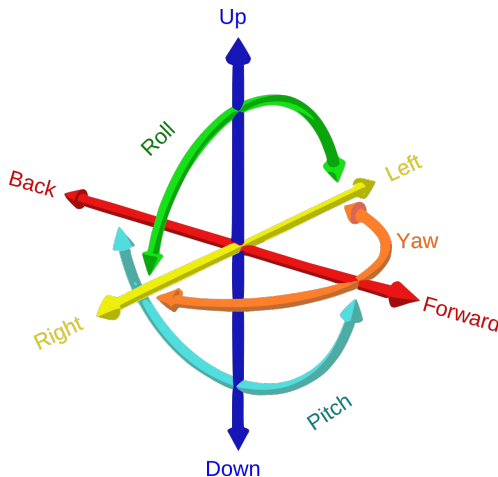
## Contents

- Introduction to SLAM
- **Key Concepts of 3D Graph SLAM**
- Graph SLAM Optimization
- SLAM Optimization in 2D
- SLAM Optimization in 3D



## Representation in 3D Space

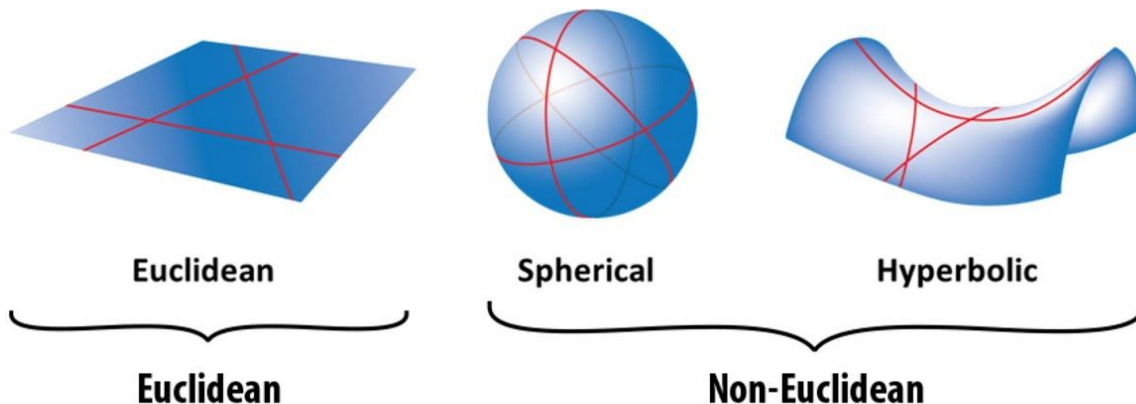
- Three correspond to rotational movement around the x, y, and z axes, commonly termed **roll**, **pitch**, and **yaw**.
- The other three correspond to translational movement along those axes, which can be thought of as moving **forward** or **backward**, moving **left** or **right**, and moving **up** or **down**.
- 6 degrees of freedom





## Non-Euclidean Spaces

A space in which the rules of Euclidean space don't apply is called **non-Euclidean space**.





## Manifolds (1)

We need a concept to deal with operations in non-Euclidean spaces:

- One approach is to represent orientation using **Euler Angles**. However, they are not perfect. There are certain orientation that can lead to a singularity called “**gimbal lock**”.
- Another approach is to over-parameterise the variables using **quaternions** and then normalise it. While this avoids gimbal locks, they introduce a new challenge. Overparameterization can lead to ambiguity (multiple parameter sets for a single rotation), problematic for applications requiring unique solutions.

To solve these problems:

- a combination of both approaches can be used.



## Manifolds (2)

- Manifolds provide a way to bridge the gap between the global expressiveness of overparameterization and the local efficiency of minimal representations.
- Informally, a manifold is a mathematical space that is non-necessarily Euclidean on a global scale, but can be seen as euclidean on a local scale.

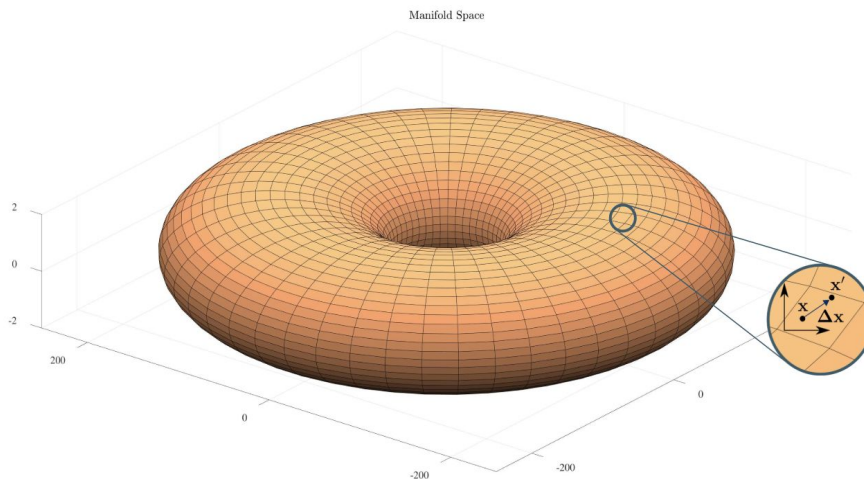


Fig: Illustration of a Manifold space. Since the manifold is smooth, local perturbations - i.e.  $\Delta x$  in the illustration - can be expressed with a suitable Euclidean vector.



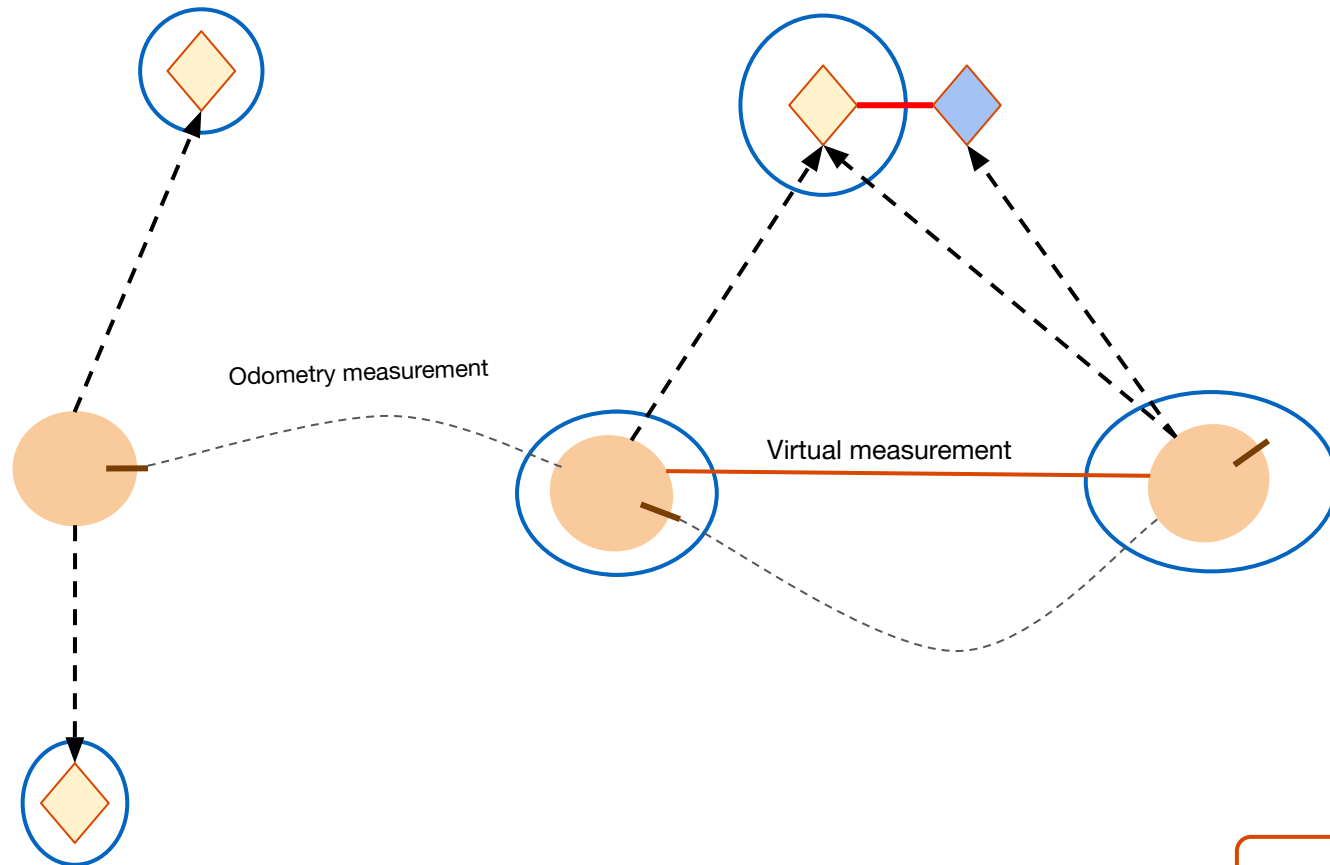
## Box Plus ( $\boxplus$ ) and Box Minus ( $\boxminus$ ) Operators

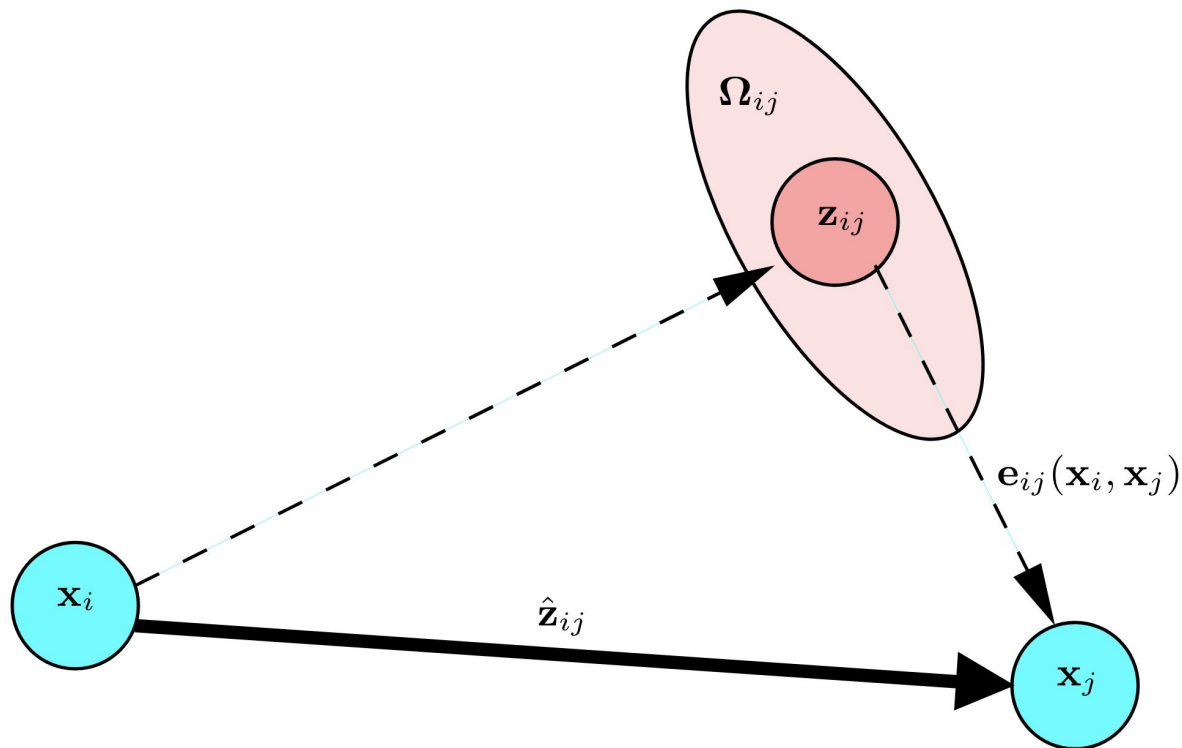
- The handling of manifolds can be encapsulated, using:
  - **Box Plus:** add a small change to a point on the manifold  
 $\boxplus : M \times \mathbb{R}^m \rightarrow M$
  - **Box Minus:** find the difference between two points on manifold elements:  $\boxminus : M \times M \rightarrow \mathbb{R}^m$
  - **Formally**, for all  $x, y \in M$ 
$$x \boxplus (y \boxminus x) = y$$



## Contents

- Introduction to SLAM
- Key Concepts of 3D Graph SLAM
- **Graph SLAM Optimization**
- SLAM Optimization in 2D
- SLAM Optimization in 3D







## Contents

- Introduction to SLAM
- Key Concepts of 3D Graph SLAM
- Graph SLAM Optimization
- **SLAM Optimization in 2D**
- SLAM Optimization in 3D



## Least Squares Approach

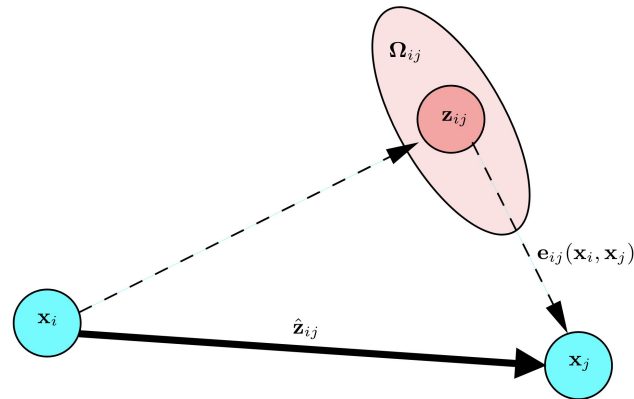
- **Goal:** to find the configuration of nodes  $\mathbf{x}^*$  that minimizes the negative log likelihood  $F(\mathbf{x})$  of all the observations

$$F(\mathbf{x}) = \sum_{\langle i,j \rangle \in \mathbb{C}} \mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}$$

- which seeks to solve

$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}} F(\mathbf{x})$$

- We will now describe an approach to solve the above equation, known as the **Gauss-Newton Algorithm**.





## Gauss-Newton Method in 2D

- Iterative minimization of

$$F(x) = \sum_{\langle i,j \rangle \in \mathbb{C}} e_{ij}^T \Omega_{ij} e_{ij}$$

- Each iteration refines the current estimate by applying a perturbation

$$x^* \leftarrow x^* + \Delta x$$

- The perturbation obtained by minimizing a quadratic approximation of the problem in  $\Delta x$  is:

$$F(x + \Delta x) \approx \Delta x^T H \Delta x + 2b^T \Delta x + c$$



## Gauss-Newton Method in 2D

$$F(x^* + \Delta x) \approx \Delta x^T \mathbf{H} \Delta x + 2 \mathbf{b}^T \Delta x + \mathbf{c}$$

$$\mathbf{H} = \sum_i \mathbf{H}^{[i]} = \sum_i \mathbf{J}^{[i]T} \mathbf{\Omega}^{[i]} \mathbf{J}^{[i]},$$

$$\mathbf{b}^T = \sum_i \mathbf{b}^{[i]T} = \sum_i \mathbf{J}^{[i]T} \mathbf{\Omega}^{[i]} \mathbf{e}^{[i]},$$

$$\mathbf{c} = \sum_i \mathbf{c}^{[i]} = \sum_i \mathbf{e}^{[i]T} \mathbf{\Omega}^{[i]} \mathbf{e}^{[i]},$$



## Gauss-Newton Method in 2D

- To determine the solution of the optimization problem we find the  $\Delta x$  that nullifies the derivative of the quadratic form  $F(\Delta x)$

$$\frac{\partial F(\Delta x)}{\partial \Delta x} = 2 \Delta x^T H + 2b^T = 0$$

$$H \Delta x^* = -b$$

- Linearised solution is then obtained by adding to the initial guess

$$x^* = x^* + \Delta x^*$$



**Algorithm:** Computing the optimum  $x^*$  of the multivariate gaussian approximation of the robot pose posterior from a graph of constraints given by  $Z$

**Input:** Initial guess  $x^*$  and measurements  $Z = \left\{ \left( z_{ij}, \Omega_{ij} \right) \right\}$

**while**  $\neg$  "termination condition" do

$H \leftarrow O$

$b \leftarrow O$

**for all**  $(z_{ij}, \Omega_{ij}) \in Z$  do

$$e_{ij} \leftarrow h_{ij}(x^*) - z_{ij}$$

$$J_{ij} \leftarrow \left. \frac{\partial e_{ij}(x)}{\partial x} \right|_{x=x^*}$$

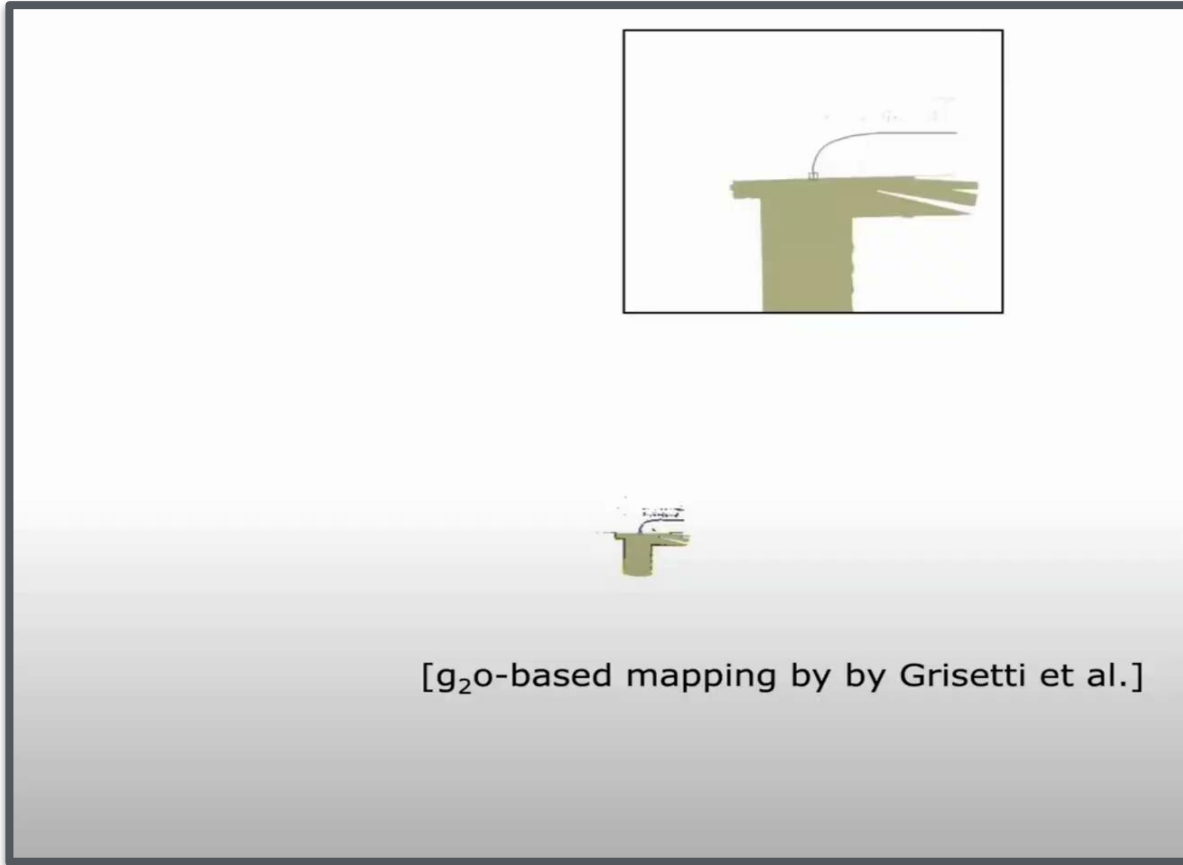
$$H \leftarrow H + J_{ij}^T \Omega_{ij} J_{ij}$$

$$b^T \leftarrow b^T + J_{ij}^T \Omega_{ij} e_{ij}$$

$$\Delta x^* \leftarrow \text{solve}(H \Delta x = -b)$$

$$x^* \leftarrow x^* + \Delta x^*$$

**return**  $x^*$





## Contents

- Introduction to SLAM
- Key Concepts of 3D Graph SLAM
- Graph SLAM Optimization
- SLAM Optimization in 2D
- **SLAM Optimization in 3D**



## Algorithm: Gauss Newton in 2D

**Note:**

**Be careful of the +  
and - operators**

**Input:** Initial guess  $x^*$  and measurements

**while**  $\neg$  "termination condition" do

$H \leftarrow O$

$b \leftarrow O$

**for all**  $(z_{ij}, \Omega_{ij}) \in Z$  do

$$e_{ij} \leftarrow h_{ij}(x^*) - z_{ij}$$

$$J_{ij} \leftarrow \left. \frac{\partial e_{ij}(x)}{\partial x} \right|_{x=x^*}$$

$$H \leftarrow H + J_{ij}^T \Omega_{ij} J_{ij}$$

$$b^T \leftarrow b^T + J_{ij}^T \Omega_{ij} e_{ij}$$

$$\Delta x^* \leftarrow \text{solve}(H \Delta x = -b)$$

$$x^* \leftarrow x^* + \Delta x^*$$

**return**  $x^*$

$$Z = \left\{ (z_{ij}, \Omega_{ij}) \right\}$$

Error should be in a vector space, as we do matrix update for b

Derivative around a manifold object?? - Impossible

Solving a linear equation would return us a vector space object which contains configuration of nodes

Adding a vector perturbation to a manifold object simply using addition?



## Gauss-Newton in Non-Euclidean Spaces

- Relaxing the assumptions that our measurements are in euclidean spaces.
- Error Function

$$e_{ij}(x) = H_{ij}(X^*) \ominus Z_{ij}$$

- Taylor Expansion:

$$e_{ij}(X \boxplus \Delta x) = \underbrace{e_{ij}(X)}_{e_{ij}} + \underbrace{\frac{\partial e_{ij}(X \boxplus \Delta x)}{\partial \Delta x}}_{J_{ij}} \bigg|_{\Delta x=0} \Delta x$$

- Increments

$$X \leftarrow X \boxplus \Delta x$$



## Algorithm: Manifold least Squares Gauss Newton Algorithm

**Input:** Initial guess  $x^*$  and measurements  $Z = \left\{ \left( z_{ij}, \Omega_{ij} \right) \right\}$

**while**  $\neg$  "termination condition" **do**

$H \leftarrow O$

$b \leftarrow O$

**for all**  $\left( z_{ij}, \Omega_{ij} \right) \in Z$  **do**

$$e_{ij} \leftarrow h_{ij} \left( X^* \right) \boxminus Z_{ij}$$

Box minus that deals with  
measurement space

$$J_{ij} \leftarrow \left. \frac{\partial e_{ij} \left( X^* \boxplus \Delta x \right)}{\partial \Delta x} \right|_{\Delta x=0}$$

Box plus that deals with  
state space

$$H \leftarrow H + J_{ij}^T \Omega_{ij} J_{ij}$$

$$b^T \leftarrow b^T + J_{ij}^T \Omega_{ij} e_{ij}$$

$$\Delta x \leftarrow \text{solve} \left( H \Delta x = -b \right)$$

$$X^* \leftarrow X^* \boxplus \Delta x$$

Box plus deals with adding  
perturbation to manifold

**return**  $X^*$



## A 3D SLAM Example

- A robot navigates an unknown environment with  $M$  landmarks. It is capable of perceiving the 3D position of the landmarks through a laser sensor. The robot also has a calibrated wheel encoder that enables odometry measurements.
- With the landmarks measurements and  $N$  odometry, estimate the robot's most likely trajectory, and the landmarks' positions (map)!



## 3D SLAM Solution Approach

- State:**  $\mathbf{X} : \mathbf{X} = \{\mathbf{X}_r^{[1]}, \dots, \mathbf{X}_r^{[N]}, \mathbf{X}_l^1, \dots, \mathbf{X}_l^{[M]}\}$

robot

$$\mathbf{X}_r^{[n]} \in SE(3) : \mathbf{X}^{[n]} = (\mathbf{R}^{[n]} | \mathbf{t}^{[n]})$$

landmark

$$\mathbf{X}_l^{[m]} \in \mathbb{R}^3 : \mathbf{X}_l^{[m]} = (x^{[m]} \ y^{[m]} \ z^{[m]})^T$$

- Increments/ Small perturbation for each state variable:**

$$\Delta \mathbf{x} \in \mathbb{R}^{6N+3M} : \Delta \mathbf{x} = \left( \Delta \mathbf{x}_r^{[1]T}, \dots, \Delta \mathbf{x}_r^{[N]T}, \Delta \mathbf{x}_l^{[1]T}, \dots, \Delta \mathbf{x}_l^{[M]T} \right)^T$$

robot

$$\Delta \mathbf{x}_r^{[n]T} \in \mathbb{R}^6 : \Delta \mathbf{x}_r^{[n]T} = \underbrace{(\Delta x^{[n]} \ \Delta y^{[n]} \ \Delta z^{[n]})}_{\Delta \mathbf{t}^{[n]}} \underbrace{(\Delta \alpha_x^{[n]} \ \Delta \alpha_y^{[n]} \ \Delta \alpha_z^{[n]})'}_{\Delta \alpha^{[n]}}^T$$

landmark

$$\Delta \mathbf{x}_l^{[m]T} \in \mathbb{R}^3 : \Delta \mathbf{x}_l^{[m]T} = \underbrace{(\Delta x^{[m]} \ \Delta y^{[m]} \ \Delta z^{[m]})^T}_{\Delta \mathbf{t}^{[m]}}$$



## 3D SLAM Solution Approach

- **Adding Perturbations:**

robot

$$\begin{aligned}\mathbf{X}_r^{[n]'} &= \mathbf{X}_r^{[n]} \boxplus \Delta \mathbf{x}_r^{[n]} \\ &= v2t(\Delta \mathbf{x}_r^{[n]}) \mathbf{X}_r^{[n]}\end{aligned}$$

landmark

$$\mathbf{X}_l^{[m]'} = \mathbf{X}_l^{[m]} + \Delta \mathbf{x}_l^{[n]}$$

- **Measurements:**

robot measurement

$$\mathbf{Z}^{[i,j]} \in SE(3) \quad : \quad \mathbf{Z}^{[i,j]} = (\mathbf{R}^{[i,j]} | \mathbf{t}^{[i,j]})$$

landmark measurement

$$\mathbf{z}^{[n,m]} \in \Re^3 \quad : \quad \mathbf{z}^{[n,m]} = (x^{[n,m]} \ y^{[n,m]} \ z^{[n,m]})^T$$



## 3D SLAM Solution Approach

- **Measurement Functions:**

robot-robot

$$\mathbf{h}^{[i,j]}(\mathbf{X}) = \mathbf{X}_r^{[i]-1} \mathbf{X}_r^{[j]}$$

landmark-robot

$$\mathbf{h}^{[n,m]}(\mathbf{X}) = \mathbf{X}_r^{[n]-1} \mathbf{X}_l^{[m]}$$

- **Errors and Jacobians.....**



## Error Function and Jacobian: Landmark-Robot Pose

$$\begin{aligned} \mathbf{e}^{[n,m]}(\mathbf{X}) &= \mathbf{X}_r^{[n]} \mathbf{X}_l^{[m]} - \mathbf{z}^{[n,m]} \\ \mathbf{e}^{[n,m]}(\mathbf{X} \boxplus \Delta \mathbf{x}) &= v2t(\Delta \mathbf{x}_r^{[n]}) \mathbf{X}_r^{[n]} (\mathbf{X}_l^{[m]} + \Delta \mathbf{x}_l^{[m]}) - \mathbf{z}^{[n,m]} \end{aligned}$$

$$\begin{aligned} \left. \frac{\partial \mathbf{e}^{[n,m]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[n]}} \right|_{\Delta \mathbf{x}_r^{[n]}=0} &= \underbrace{\left( \mathbf{I}_{3 \times 3} \mid \lfloor -\hat{\mathbf{z}}^{[m,n]} \rfloor_{\times} \right)}_{\mathbf{J}_r^{n,m}} \\ \left. \frac{\partial \mathbf{e}^{[n,m]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_l^{[m]}} \right|_{\Delta \mathbf{x}_l^{[m]}=0} &= \underbrace{\mathbf{R}^{[n]}}_{\mathbf{J}_l^{n,m}} \end{aligned}$$

$$\mathbf{J}^{[n,m]} = \begin{pmatrix} \cdots & \mathbf{0}_{3 \times 6} & \cdots & \mathbf{J}_r^{[n,m]} & \cdots & \mathbf{0}_{3 \times 3} & \cdots & \mathbf{J}_l^{[n,m]} & \cdots & \mathbf{0}_{3 \times 3} & \cdots \end{pmatrix}$$



## Error Function and Jacobian: Pose-Pose

- Error Function:**

$$\mathbf{e}^{[i,j]}(\mathbf{X}) = \mathbf{X}_r^{[i]-1} \mathbf{X}_r^{[j]} \ominus \mathbf{Z}^{[i,j]}$$

$$\begin{aligned} \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x}) &= t2v \left( \mathbf{Z}^{[i,j]-1} \left( v2t(\Delta \mathbf{x}_r^{[i]}) \mathbf{X}_r^{[i]} \right)^{-1} \left( v2t(\Delta \mathbf{x}_r^{[j]}) \mathbf{X}_r^{[j]} \right) \right) \\ &= t2v \left( \mathbf{Z}^{[i,j]-1} \mathbf{X}_r^{[i]-1} v2t(\Delta \mathbf{x}_r^{[i]})^{-1} v2t(\Delta \mathbf{x}_r^{[j]}) \mathbf{X}_r^{[j]} \right) \end{aligned}$$

- Jacobian:**

$$\begin{aligned} \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}} &= \left( \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[i]}} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[j]}} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \right) \\ \left. \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[i]}} \right|_{\Delta \mathbf{x}_r^{[i]}=0} &= \mathbf{J}_i^{[i,j]} \\ \left. \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[j]}} \right|_{\Delta \mathbf{x}_r^{[j]}=0} &= \mathbf{J}_j^{[i,j]} \\ \mathbf{J}^{[i,j]} &= \left( \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \mathbf{J}_i^{[i,j]} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \mathbf{J}_j^{[i,j]} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \right) \end{aligned}$$

Not easy to compute  
with pen and paper



## Error Function and Jacobian: Pose-Pose

- The t2v function in the error is highly non-linear. So to calculate Jacobian we simplify the problem using....

## CHORDAL DISTANCE

- **Definition:** Given two transformation matrices, the chordal distance is the difference between
  - Each vector in Rotation matrix
  - The translation vectors
- 12 x 1 vector!!



## Error Function and Jacobian: Pose-Pose

How to use this in our calculations of Jacobian?

- We introduce the “flatten” function, that turns a transformation matrix in a vector containing its components, i.e. a column vector of 12 dimensions.
- Using flattening,

$$\mathbf{e}^{[i,j]}(\mathbf{X}) = \text{flatten}(\mathbf{X}_r^{[i]-1} \mathbf{X}_r^{[j]}) - \text{flatten}(\mathbf{Z}^{[i,j]})$$

$\boxed{g(\mathbf{X})}$

$$\frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}} = \left( \dots \mathbf{0}_{12 \times 6} \dots \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^i} \dots \mathbf{0}_{12 \times 6} \dots \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^j} \dots \mathbf{0}_{12 \times 6} \dots \right)$$

$\boxed{\text{Easier to compute with pen and paper}}$



## Error Function and Jacobian: Pose-Pose

- Expanding the prediction and calculating the derivative evaluated at 0

$$\mathbf{g}(\mathbf{X}_i \boxplus \Delta \mathbf{x}_i, \mathbf{X}_j \boxplus \Delta \mathbf{x}_j) = \mathbf{X}_i^{-1} v2t(\Delta \mathbf{x}_i)^{-1} v2t(\Delta \mathbf{x}_j) \mathbf{X}_j$$

$$v2t(\Delta \mathbf{x})^{-1} \simeq v2t(-\Delta \mathbf{x}) \text{ for small } \Delta \mathbf{x}$$

$$\mathbf{g}(\mathbf{X}_i \boxplus \Delta \mathbf{x}_i, \mathbf{X}_j \boxplus \Delta \mathbf{x}_j) = \underbrace{\begin{pmatrix} \mathbf{R}_i^T & -\mathbf{R}_i^T \mathbf{t}_i \\ & 1 \end{pmatrix}}_{\mathbf{X}_i^{-1}} \underbrace{\begin{pmatrix} \mathbf{R}(-\Delta \alpha_i) & -\Delta \mathbf{t}_i \\ & 1 \end{pmatrix}}_{v2t(\Delta \mathbf{x}_i)^{-1}} \underbrace{\begin{pmatrix} \mathbf{R}(\Delta \alpha_j) & \Delta \mathbf{t}_j \\ & 1 \end{pmatrix}}_{v2t(\Delta \mathbf{x}_j)} \underbrace{\begin{pmatrix} \mathbf{R}_j & \mathbf{t}_j \\ & 1 \end{pmatrix}}_{\mathbf{X}_j}$$

$$\left. \frac{\partial \mathbf{g}(\mathbf{X}_i \boxplus \Delta \mathbf{x}_i, \mathbf{X}_j \boxplus \Delta \mathbf{x}_j)}{\partial \Delta \mathbf{x}_i} \right|_{\Delta \mathbf{x}=0} = - \left. \frac{\partial \mathbf{g}(\mathbf{X}_i \boxplus \Delta \mathbf{x}_i, \mathbf{X}_j \boxplus \Delta \mathbf{x}_j)}{\partial \Delta \mathbf{x}_j} \right|_{\Delta \mathbf{x}=0}$$

So we need to only calculate 1 and take the opposite for the other



## Error Function and Jacobian: Pose-Pose

$$\begin{aligned}
 g(\mathbf{X}_i, \mathbf{X}_j \boxplus \Delta \mathbf{x}_j) &= \begin{bmatrix} \mathbf{R}_i^T & -\mathbf{R}_i^T \mathbf{t}_i \end{bmatrix} \begin{bmatrix} \mathbf{R}(\Delta \alpha_j) & \Delta \mathbf{t}_j \end{bmatrix} \begin{bmatrix} \mathbf{R}_j & \mathbf{t}_j \end{bmatrix} \\
 &= \begin{bmatrix} \mathbf{R}_i^T \mathbf{R}(\Delta \alpha_j) \mathbf{R}_j & \mathbf{R}_i^T (\mathbf{R}(\Delta \alpha_j) \mathbf{t}_j + \Delta \mathbf{t}_j - \mathbf{t}_i) \end{bmatrix}
 \end{aligned}$$

Considering only  $\Delta \mathbf{x}_j$

$$\frac{\partial \mathbf{g}}{\partial \Delta \alpha_x} = \begin{pmatrix} \mathbf{R}_i^T \mathbf{R}'_{x0} \mathbf{R}_j & \mathbf{R}_i^T \mathbf{R}'_{x0} \mathbf{t}_j \end{pmatrix} \quad \frac{\partial \mathbf{g}}{\partial \Delta \mathbf{t}_x} = \begin{pmatrix} \mathbf{0}_{3 \times 3} & \mathbf{R}_i^T (1 \ 0 \ 0)^T \end{pmatrix}$$

$$\frac{\partial \mathbf{g}}{\partial \Delta \alpha_y} = \begin{pmatrix} \mathbf{R}_i^T \mathbf{R}'_{y0} \mathbf{R}_j & \mathbf{R}_i^T \mathbf{R}'_{y0} \mathbf{t}_j \end{pmatrix} \quad \frac{\partial \mathbf{g}}{\partial \Delta \mathbf{t}_y} = \begin{pmatrix} \mathbf{0}_{3 \times 3} & \mathbf{R}_i^T (0 \ 1 \ 0)^T \end{pmatrix}$$

$$\frac{\partial \mathbf{g}}{\partial \Delta \alpha_z} = \begin{pmatrix} \mathbf{R}_i^T \mathbf{R}'_{z0} \mathbf{R}_j & \mathbf{R}_i^T \mathbf{R}'_{z0} \mathbf{t}_j \end{pmatrix} \quad \frac{\partial \mathbf{g}}{\partial \Delta \mathbf{t}_z} = \begin{pmatrix} \mathbf{0}_{3 \times 3} & \mathbf{R}_i^T (0 \ 0 \ 1)^T \end{pmatrix}$$



## Error Function and Jacobian: Pose-Pose

- Final Jacobian is assembled in a 12x6 matrix, by flattening the contribution of the components

$$dh_x = \text{flatten} \left( \frac{\partial g}{\partial \Delta t_x} \right)$$

$$\left. \frac{\partial h(\mathbf{X}_i, \mathbf{X}_j; \boxplus \Delta \mathbf{X}_j)}{\partial \Delta \mathbf{X}_j} \right|_{\Delta \mathbf{X}=0} = (dh_x \quad dh_y \quad dh_z \quad dh_{\alpha x} \quad dh_{\alpha y} \quad dh_{\alpha z})$$


- Variable Assignments:**

$$r'_x = \text{flatten} (\mathbf{R}_i^T \mathbf{R}'_{x0} \mathbf{R}_j)$$

$$r'_y = \text{flatten} (\mathbf{R}_i^T \mathbf{R}'_{y0} \mathbf{R}_j)$$

$$r'_z = \text{flatten} (\mathbf{R}_i^T \mathbf{R}'_{z0} \mathbf{R}_j)$$



## Error Function and Jacobian: Pose-Pose

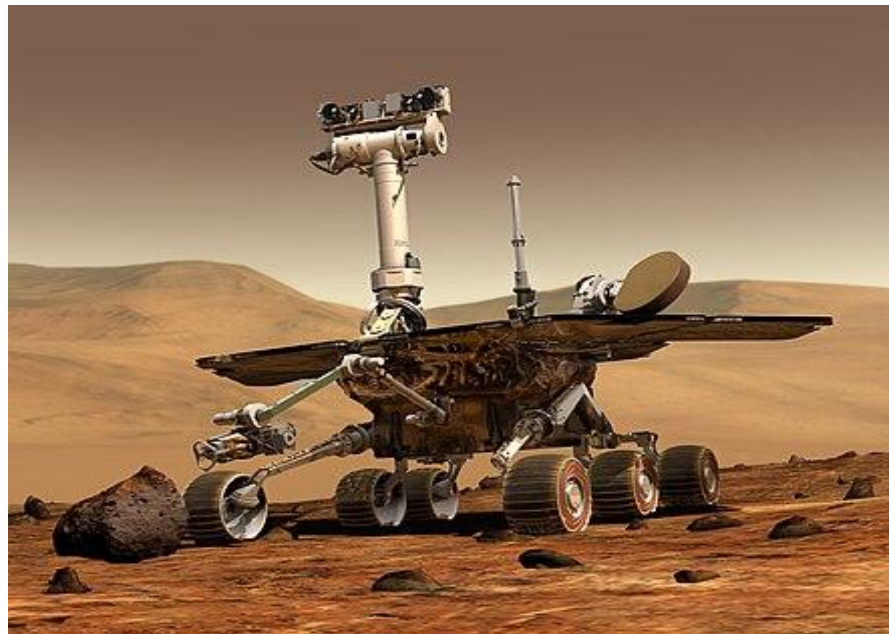
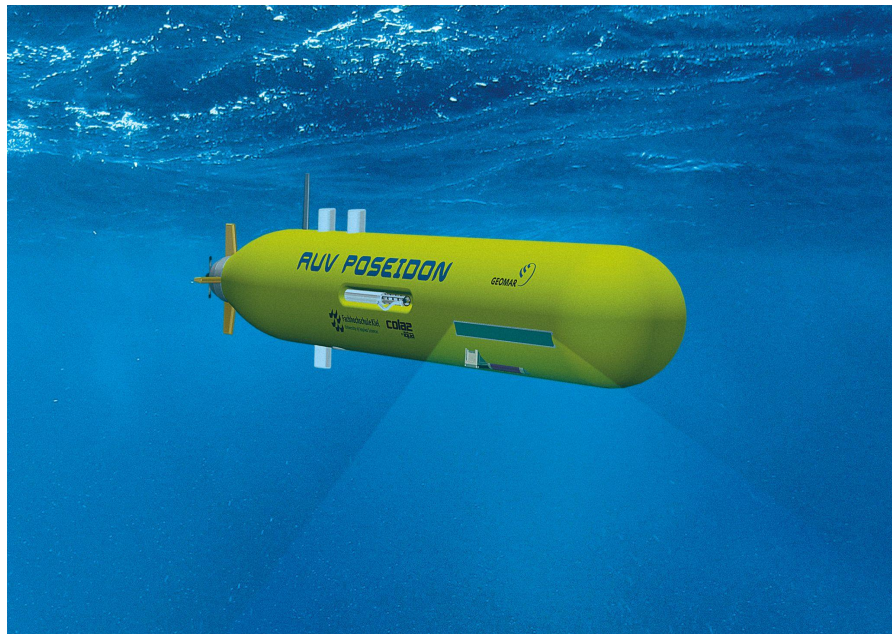
$$J_j^{[i,j]}(\mathbf{X}) = \begin{bmatrix} \mathbf{0}_{9 \times 3} & \begin{bmatrix} r'_x & r'_y & r'_z \end{bmatrix} \\ \mathbf{R}_i^T & -\mathbf{R}_i^T [\mathbf{t}_j]_{\times} \end{bmatrix}$$

12x6

$$J_i^{[i,j]}(\mathbf{X}) = -J_j^{[i,j]}(\mathbf{X})$$



## Some practical Applications





## References

- G. Grisetti, R. Kümmerle, C. Stachniss and W. Burgard, "A Tutorial on Graph-Based SLAM", ITSM 2010.
- G. Grisetti, T. Guadagnino, I. Aloise, M. Colosi, B. Della Corte, D. Schlegel, "Least Squares Optimization: From Theory to Practice", Robotics 2020.
- Giorgio Grisetti, PR\_2021\_22\_manifolds. [\[Video\]](#)
- SLAM-Course - 01 - Introduction to Robot Mapping (2013/14; Cyrill Stachniss) [\[Video\]](#)
- SLAM Using Pose Graph Optimization Wolfram Burgard, Giorgio Grisetti, and Cyrill Stachniss: Graph-based SLAM in 20 Minutes [\[Video\]](#)
- C. Hertzberg. A framework for sparse, non-linear least squares problems on manifolds. Master's thesis, Univ. Bremen, 2008.



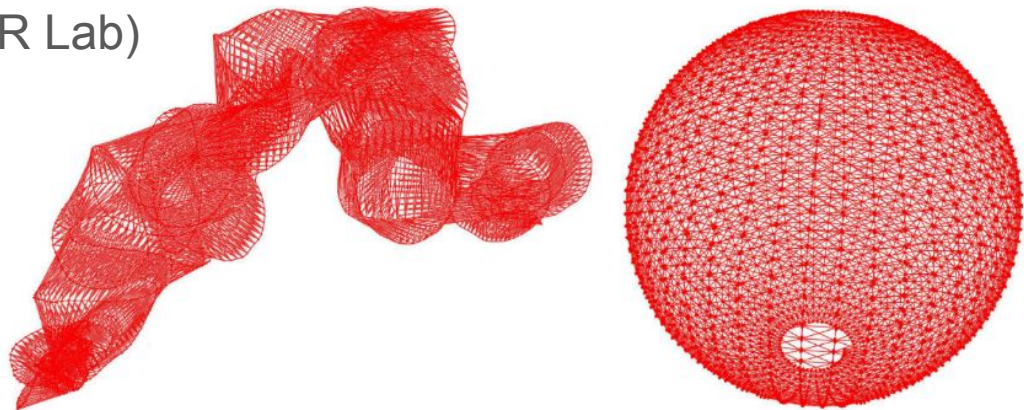
# Mobile Robotics

## 3D Graph SLAM (Implementation & Results)

---

Socially Intelligent Robot (SIR Lab)  
University of Stuttgart  
Prof. Dr. Kai Arras

Annie Bhalla, 3638974  
Likhith Jain, 3678905





## Contents

- Recap
- Graph SLAM Optimization
- Results
- Q&A



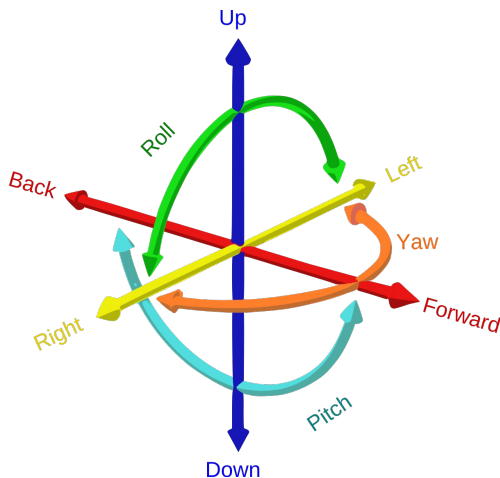
## Contents

- **Recap**
- Graph SLAM Optimization
- Results
- Q&A



## Representation in 3D Space

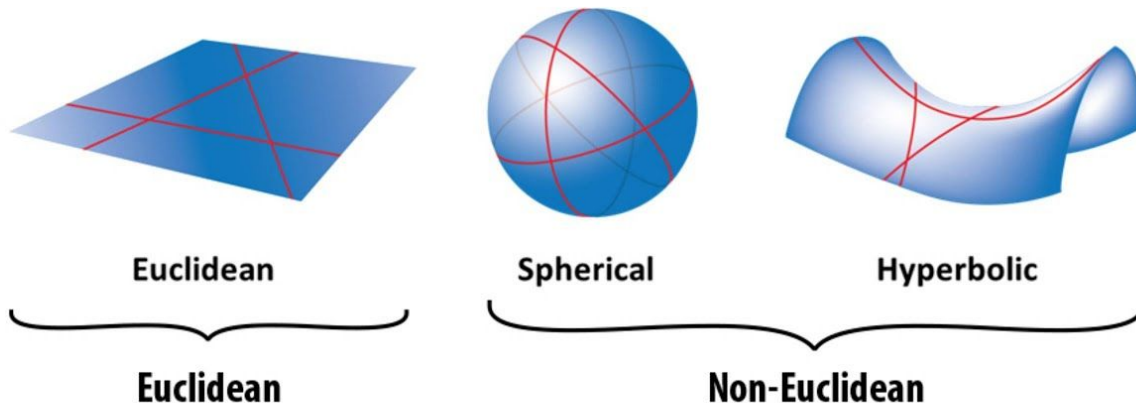
- Three correspond to rotational movement around the x, y, and z axes, commonly termed **roll**, **pitch**, and **yaw**.
- The other three correspond to translational movement along those axes, which can be thought of as moving **forward** or **backward**, moving **left** or **right**, and moving **up** or **down**.
- 6 degrees of freedom





## Non-Euclidean Spaces

A space in which the rules of Euclidean space don't apply is called **non-Euclidean space**.



# Manifolds

A manifold is a mathematical space that is non-necessarily Euclidean on a global scale, but can be seen as euclidean on a local scale.

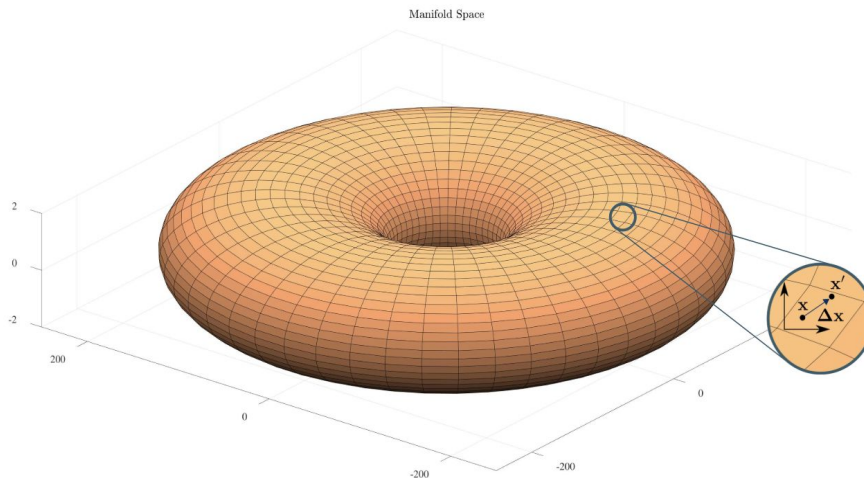


Fig: Illustration of a Manifold space. Since the manifold is smooth, local perturbations - i.e.  $\Delta x$  in the illustration - can be expressed with a suitable Euclidean vector.



## Box Plus ( $\boxplus$ ) and Box Minus ( $\boxminus$ ) Operators

- The handling of manifolds can be encapsulated, using:
  - **Box Plus:** add a small change to a point on the manifold  
 $\boxplus : M \times \mathbb{R}^m \rightarrow M$
  - **Box Minus:** find the difference between two points on manifold elements:  $\boxminus : M \times M \rightarrow \mathbb{R}^m$
  - **Formally**, for all  $x, y \in M$ 
$$x \boxplus (y \boxminus x) = y$$



## Contents

- Recap
- **Graph SLAM Optimization**
- Results
- Q&A

```
graph LR; A[Graph SLAM Optimization] --> B[Euclidean Optimization]; A --> C[Manifold Optimization];
```

Euclidean Optimization

Manifold Optimization



## Landmark-Robot Pose Optimization

- Calculating Jacobians for landmark-pose is much easier in both cases (Euclidean, and Manifold) since measurements are in  $\mathbb{R}^3$
- There is no need for special operations like box plus to handle non-Euclidean environments.



## Landmark-Robot Pose Optimization in Euclidean Space

```
function [e, Jr, Jl] =errorAndJacobianEuclideanLP(XL, XR, z)
```

```
    t= XR(1:3);
```

```
    rx = Rx(XR(4));
```

```
    ry = Ry(XR(5));
```

```
    rz = Rz(XR(6));
```

Extracting rotation and translation from XR:

```
    rx_p= Rx_prime(XR(4));
```

```
    ry_p = Ry_prime(XR(5));
```

```
    rz_p = Rz_prime(XR(6));
```

```
    z_hat= (rx*ry*rz)' *(XL - t);
```

Predicted landmark measurement

```
    e = z_hat- z;
```

Compute Error

```
    Jr = zeros(3,6);
```

```
    Jr(1:3,1:3)= -(rx*ry*rz)';
```

```
    Jr(:,4)= (rx_p * ry* rz)' * (XL-t);
```

```
    Jr(:,5)= (rx * ry_p* rz)' * (XL-t);
```

```
    Jr(:,6)= (rx * ry* rz_p)' * (XL-t);
```

Jacobian wrt robot pose

```
    Jl= (rx*ry*rz)';
```

Jacobian wrt landmark

```
end
```

### Input:

XL: landmark position in the world frame

XR: robot pose in the world frame

Z: observed measurement

### Output:

e: error between predicted and observed landmark

Jr: Jacobian of error wrt robot pose

Jl: Jacobian of error wrt landmark



## Landmark-Robot Pose Optimization in Manifold Space

```
function [e, Jr, Jl] =errorAndJacobianManifoldLP(XL, XR, z)
```

```
    R = XR(1:3,1:3);
```

```
    t= XR(1:3,4);
```

Extracting rotation and translation from XR:

```
    z_hat= R' *(XL - t);
```

Predicted landmark measurement

```
    e = z_hat- z;
```

Compute Error

```
    Jr = zeros(3,6);
```

```
    Jr(1:3,1:3)= eye(3);
```

```
    Jr(1:3, 4:6)= -skew(XL);
```

Jacobian wrt robot pose

```
    Jr = -R'*Jr;
```

```
    Jl= R';
```

Jacobian wrt landmark

```
end
```

### Input:

XL: landmark position in the world frame

XR: robot pose in the world frame

z: observed measurement

### Output:

e: error between predicted and observed landmark

Jr: Jacobian of error wrt robot pose

Jl: Jacobian of error wrt landmark



## Pose-Pose Optimization

For pose-pose optimization, we need to consider two scenarios:

- Non-Euclidean Spaces (Manifold Perturbations)
- Euclidean Spaces (Euclidean Perturbations)



## Pose-Pose Optimization

We look at the differences in Euclidean space and Manifold space in terms of:

- State Representations
- Perturbations
- Measurement Functions
- Error
- Jacobians



## State Representations

The state is a collection of robot poses

- Manifold

$$\mathbf{X}_r^{[n]} \in SE(3) \quad : \quad \mathbf{X}^{[n]} = (\mathbf{R}^{[n]} | \mathbf{t}^{[n]})$$

- Euclidean

$$\mathbf{X}_r^{[n]} \in \mathbb{R}^6 \quad : \quad \mathbf{X}_r^{[n]} = [x^{[n]} \ y^{[n]} \ z^{[n]} \ \alpha^{[n]} \ \beta^{[n]} \ \gamma^{[n]}]$$



## Perturbations

Deviations from the true values that cause errors in the estimation.

- Manifold

$$X \boxplus \Delta x \rightarrow \begin{bmatrix} R(\Delta\theta)R & t + \Delta t \\ 0 & 1 \end{bmatrix}$$

- Euclidean

$$X + \Delta x \rightarrow \begin{pmatrix} t + \Delta t \\ \theta + \Delta\theta \end{pmatrix}$$



## Measurement Functions

A measurement of the robot pose  $j$ , performed from robot pose  $i$ .

- Manifold

$$\mathbf{h}^{[i,j]}(\mathbf{X}) = \mathbf{X}_r^{[i]-1} \mathbf{X}_r^{[j]}$$

- Euclidean

$$\mathbf{h}^{[i,j]}(\mathbf{X}) = \begin{bmatrix} R_i^T(t_j - t_i) \\ p_i(4:6) - p_j(4:6) \end{bmatrix}$$



## Error

The error is the difference between prediction and measurement.

- Manifold

$$\mathbf{e}^{[i,j]}(\mathbf{X}) = \mathbf{X}_r^{[i]-1} \mathbf{X}_r^{[j]} \ominus \mathbf{Z}^{[i,j]}$$

- Euclidean

$$\mathbf{e}^{[i,j]}(\mathbf{X}) = (\mathbf{X}_r^{[j]} - \mathbf{X}_r^{[i]}) - \mathbf{Z}^{[i,j]}$$



## Jacobians

- Manifold

$$\begin{aligned}
 \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}} &= \left( \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[i]}} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[j]}} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \right) \\
 \left. \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[i]}} \right|_{\Delta \mathbf{x}_r^{[i]}=0} &= \mathbf{J}_i^{[i,j]} \\
 \left. \frac{\partial \mathbf{e}^{[i,j]}(\mathbf{X} \boxplus \Delta \mathbf{x})}{\partial \Delta \mathbf{x}_r^{[j]}} \right|_{\Delta \mathbf{x}_r^{[j]}=0} &= \mathbf{J}_j^{[i,j]} \\
 \mathbf{J}^{[i,j]} &= \left( \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \mathbf{J}_i^{[i,j]} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \quad \mathbf{J}_j^{[i,j]} \quad \cdots \quad \mathbf{0}_{6 \times 6} \quad \cdots \right)
 \end{aligned}$$

- Euclidean

$$\frac{\partial e^{[i,j]}}{\partial x_r^i} = \frac{Z_{\text{pred}}}{\partial x_r^i} = J_i$$

$$\frac{\partial e^{[i,j]}}{\partial x_r^j} = \frac{Z_{\text{pred}}}{\partial x_r^j} = J_j$$



## Robot Pose-Robot Pose Optimization in Euclidean Space

```
function [e,Ji,Jj]= errorAndJacobianEuclideanPP(pi, pj, Z)
```

```
% measurement function
```

```
ti = pi(1:3);  
Ri = Rx(pi(4))*Ry(pi(5))*Rz(pi(6));  
tj = pj(1:3);  
Rj = Rx(pj(4))*Ry(pj(5))*Rz(pj(6));
```

Extracting rotation and translation from pi and pj

```
Ria = Rx_prime(pi(4))*Ry(pi(5))*Rz(pi(6));  
Rib = Rx(pi(4))*Ry_prime(pi(5))*Rz(pi(6));  
Ric = Rx(pi(4))*Ry(pi(5))*Rz_prime(pi(6));
```

Derivative Rotation Matrices

```
% tj as seen from pi for the translation part  
% for the rotation part, just the difference between the angle vectors  
Z_pred = [Ri'*(tj-ti);  
          pj(4:6)-pi(4:6)];
```

Predicted Measurements

```
% Error
```

```
e = Z_pred - Z;
```

Error Calculation

```
I = eye(3,3);  
Ji = [-Ri' Ria'*(tj-ti) Rib'*(tj-ti) Ric'*(tj-ti);  
      zeros(3,3), -I];  
Jj = [Ri' zeros(3,3);  
      zeros(3,3) -I];
```

Jacobian wrt pi

Jacobian wrt pj

```
end
```

### Input:

pi: robot pose i  
Pj: robot pose j  
Z: Observed relative pose between pi and pj

### Output:

e: Error vector  
Ji: Jacobian with respect to pi  
Jj: Jacobian with respect to pj



## Robot Pose-Robot Pose Optimization in Manifold Space

```
function [e, Ji, Jj] = errorAndJacobianManifoldPP(pi, pj, Z)
```

```
Rdx = [0,0,0;  
       0,0,-1;  
       0,1,0];
```

```
Rdy = [0,0,1;  
       0,0,0;  
       -1,0,0];
```

```
Rdz = [0,-1,0;  
       1,0,0;  
       0,0,0];
```

Define Rotation Derivatives

```
Xi = pi;  
Xi_inv = inverse(Xi);  
Ri_transpose = Xi_inv(1:3,1:3);
```

```
Rj = pj(1:3,1:3);  
tj = pj(1:3,4);
```

Extracting rotation and translation from pi and pj

Calculate the partial derivatives of the rotation components of the transformation

```
rxp = Ri_transpose * Rdx * Rj;  
rxp = reshape(rxp, [9 1]);
```

```
ryp = Ri_transpose * Rdy * Rj;  
ryp = reshape(ryp, [9 1]);
```

```
rzp = Ri_transpose * Rdz * Rj;  
rzp = reshape(rzp, [9 1]);
```

Jacobian wrt pj

```
Jj = [zeros(9,3) , rxp, ryp, rzp;  
      Ri_transpose, -Ri_transpose*skew(tj)];  
Ji = -Jj;
```

Jacobian wrt pi

```
g = Xi_inv * pj;
```

```
e = flatten(g) - flatten(Z);
```

Error Calculation and Flatten

```
end
```

### Input:

pi: robot pose i in HT  
Pj: robot pose j in HT  
Z: Measured Transformation between pi and pj in HT

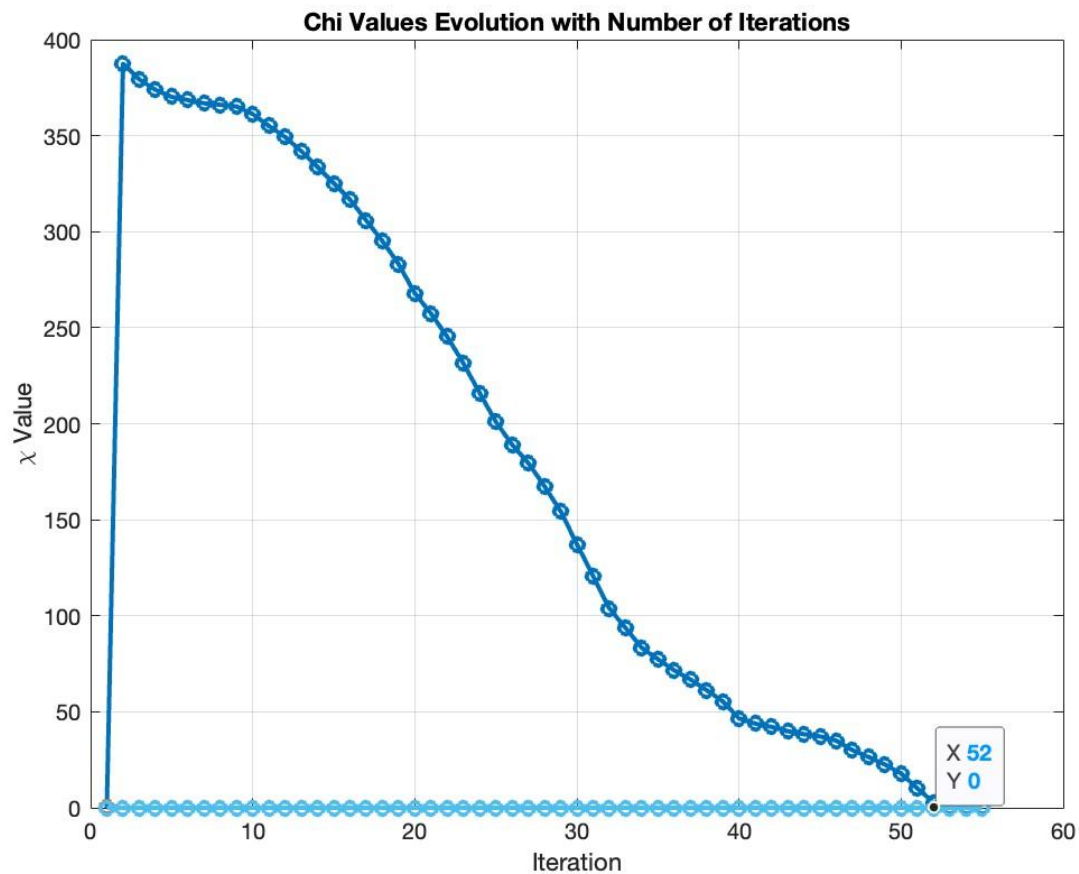
### Output:

e: Error vector  
Ji: Jacobian with respect to pi  
Jj: Jacobian with respect to pj

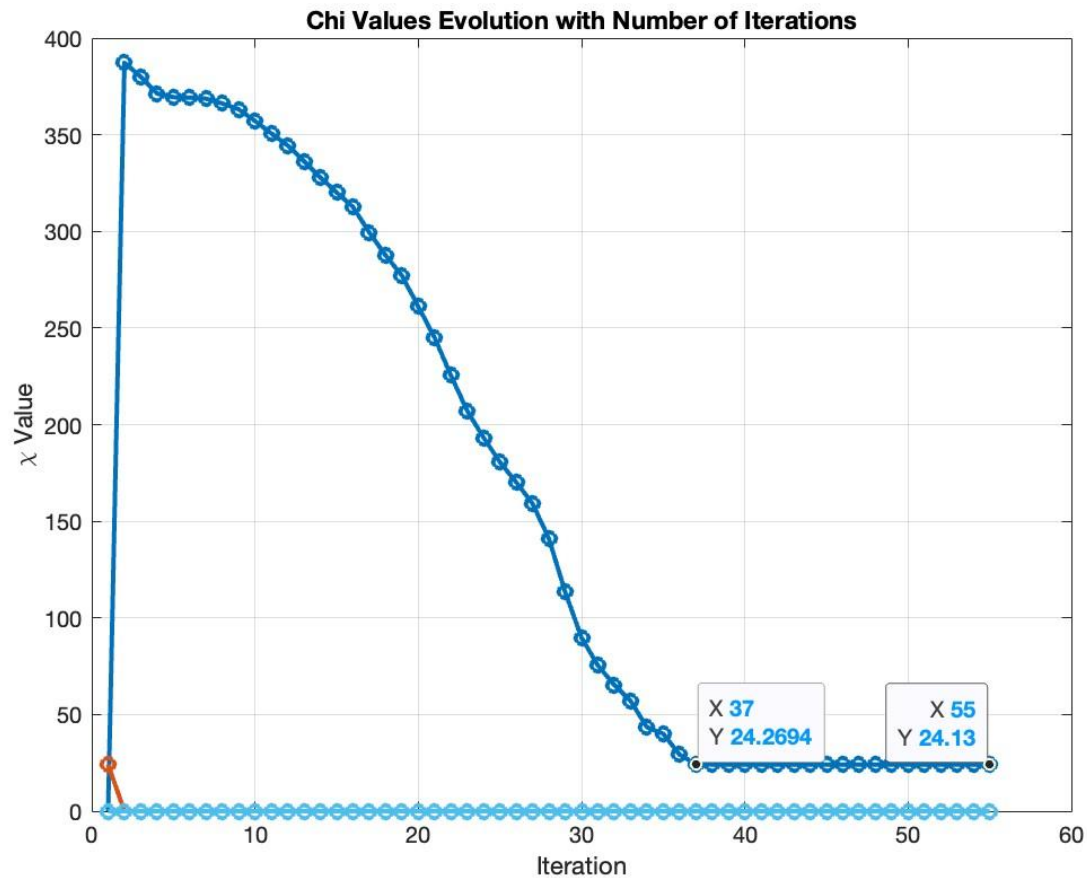


## Contents

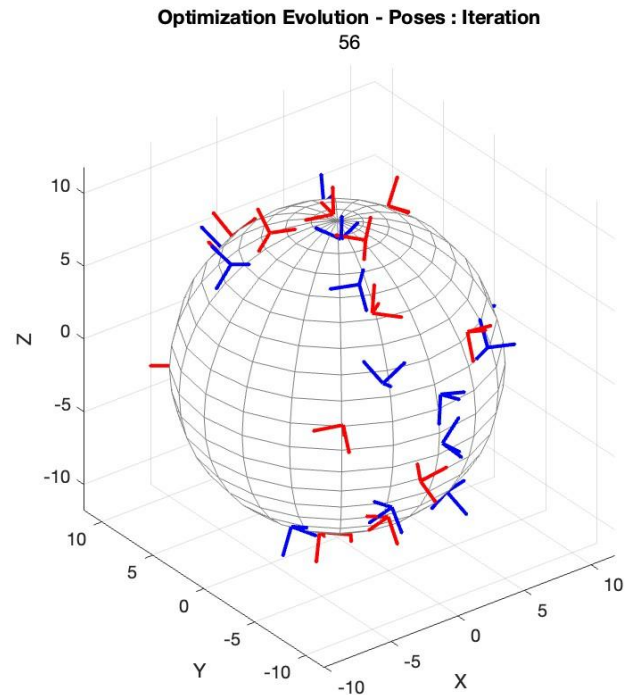
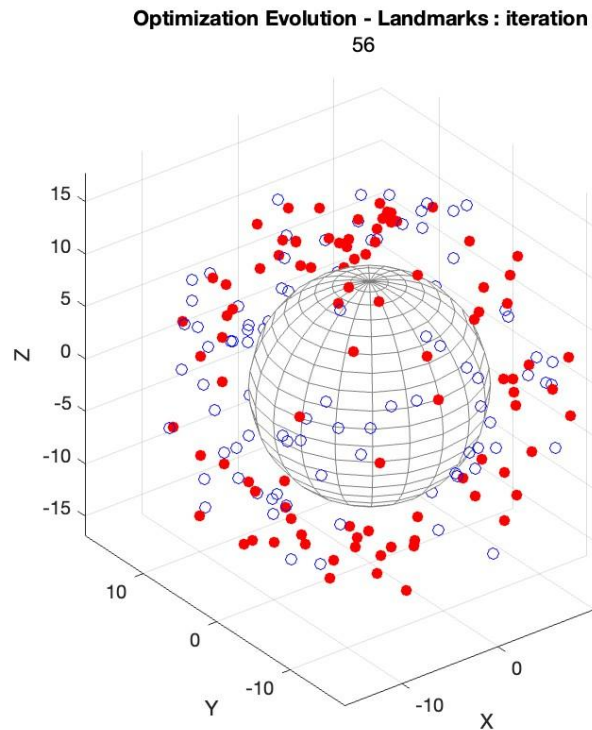
- Recap
- Graph SLAM Optimization
- **Results (MATLAB)**
- Q&A



# Euclidean Perturbation



# Trajectory with singularities - Euclidean Optimization

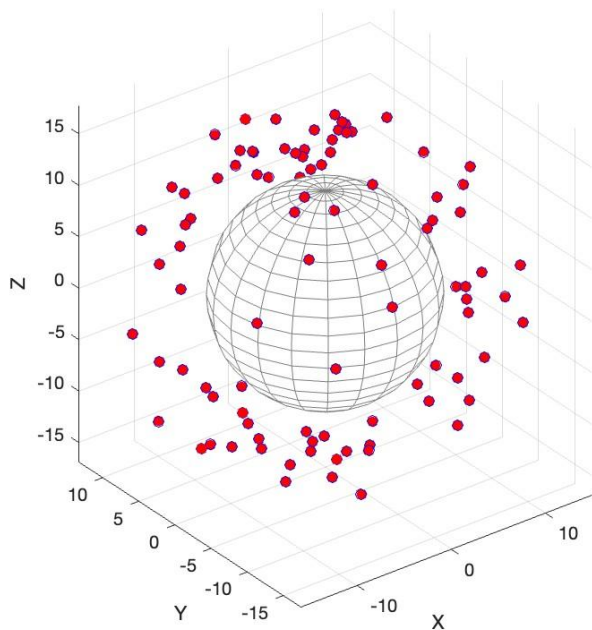


# Trajectory with singularities - Manifold Optimization



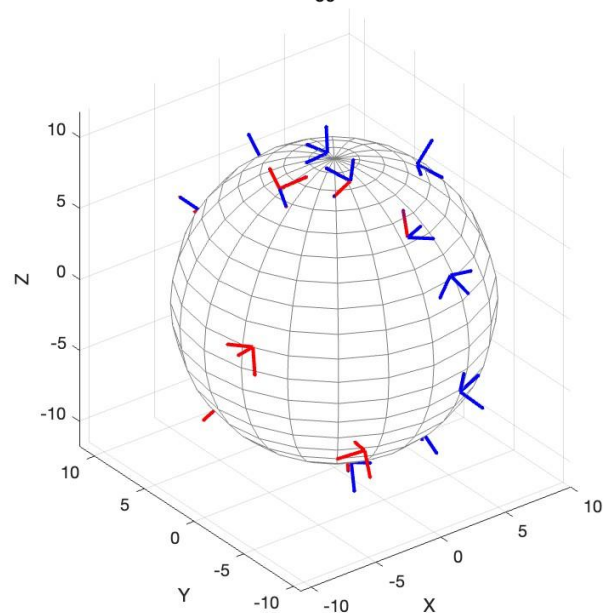
Optimization Evolution - Landmarks : iteration

56



Optimization Evolution - Poses : Iteration

56





## Contents

- Recap
- Graph SLAM Optimization
- Results
- **Q&A**